



Машинное обучение

Лекция № 16

Введение в нейронные сети

Докладчик: Ваня Горохов



Многослойный перцептрон

Основа большинства современных нейросетей — это линейный слой, преобразование вида:

$$\text{Linear}(x) = W \cdot x + b$$

Запись $\text{Linear}(d_{in}, d_{out})$ означает, что матрица $W \in \mathbb{R}^{d_{out} \times d_{in}}$, а вектор $b \in \mathbb{R}^{d_{out}}$.

Многослойный перцептрон (MLP) — это модель следующего вида:

$$\begin{aligned} h_0 &= x \\ h_i &= \sigma(\text{Linear}_i(h_{i-1})) \\ f(x) &= \text{Linear}_N(h_{N-1}) \end{aligned}$$

- Первый слой:
 $\text{Linear}_1(d_{in}, d_{hidden})$
- Скрытые слои ($1 < i < N$):
 $\text{Linear}_i(d_{hidden}, d_{hidden})$
- Выходной слой:
 $\text{Linear}_N(d_{hidden}, d_{out})$



Конфигурация MLP

Архитектура MLP определяется параметрами, которые можно разделить на два типа:

- **Гиперпараметры:**
 - Количество скрытых слоев — глубина сети.
 - Размерность скрытого представления d_{hidden} — ширина сети.
- **Параметры задачи:**
 - Входная размерность d_{in} — число признаков.
 - Выходная размерность d_{out} , которая зависит от того, что мы предсказываем.

Выбор d_{out} определяется задачей:

- **Регрессия:** $d_{out} = 1$.
- **Множественная регрессия:** $d_{out} = K$.
- **Многоклассовая классификация (C классов):** $d_{out} = C$. Дополнительно применяется **Softmax**:

$$\text{Softmax}(z)_i = \frac{e^{z_i}}{\sum_{j=1}^C e^{z_j}}$$

Преобразует сырые выходы (логиты) в распределение вероятностей:

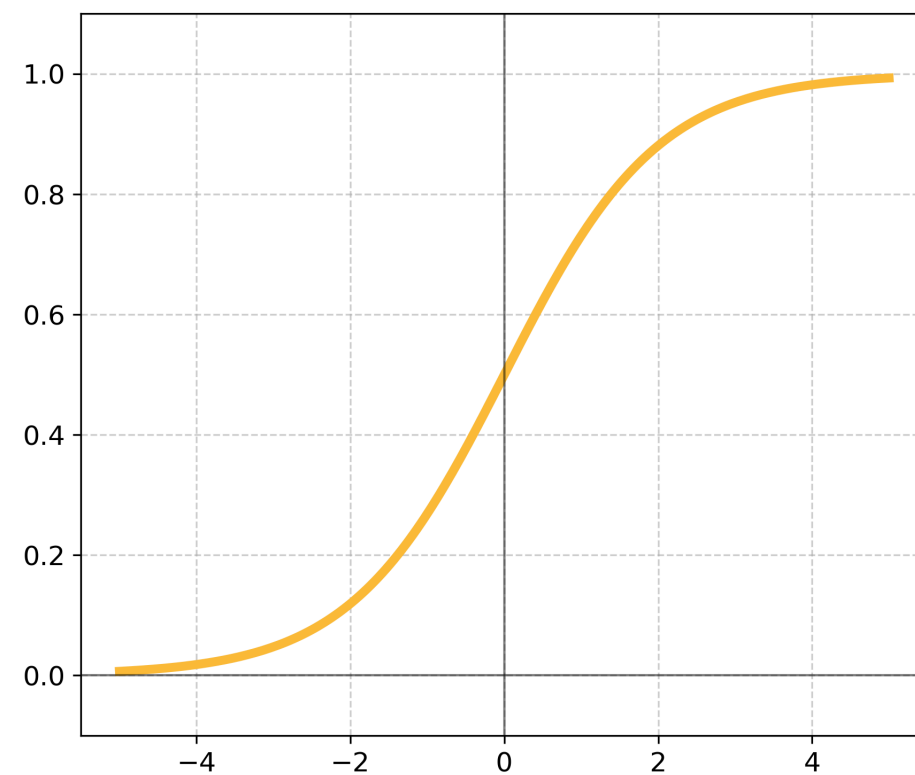
- Каждое значение > 0
- Сумма всех значений = 1



Популярные функции активации

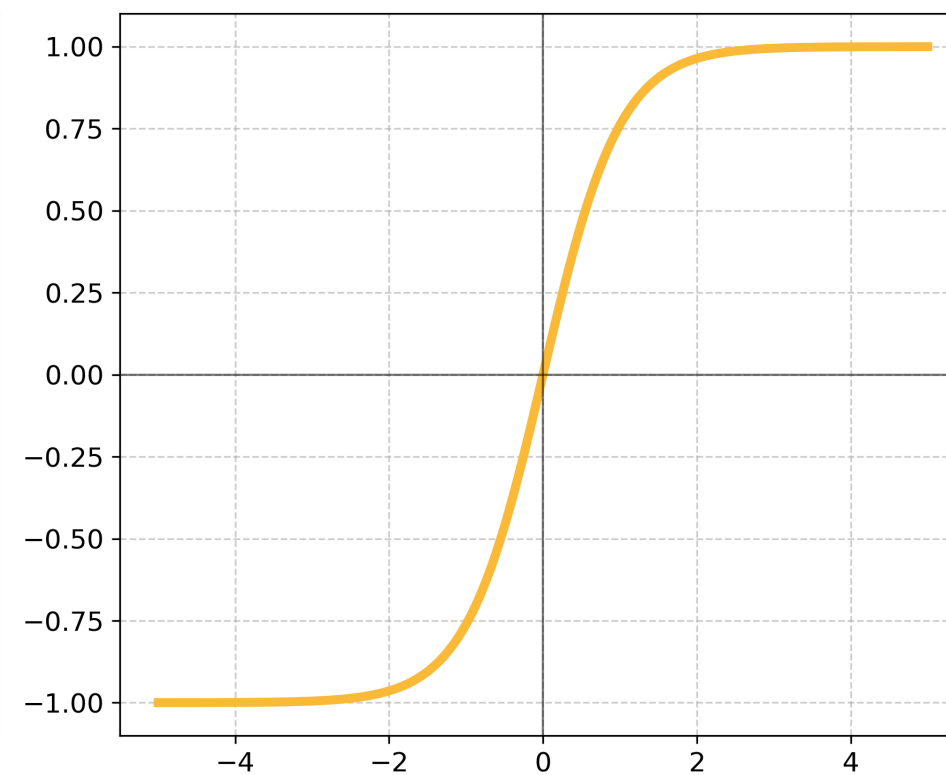
Sigmoid

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$



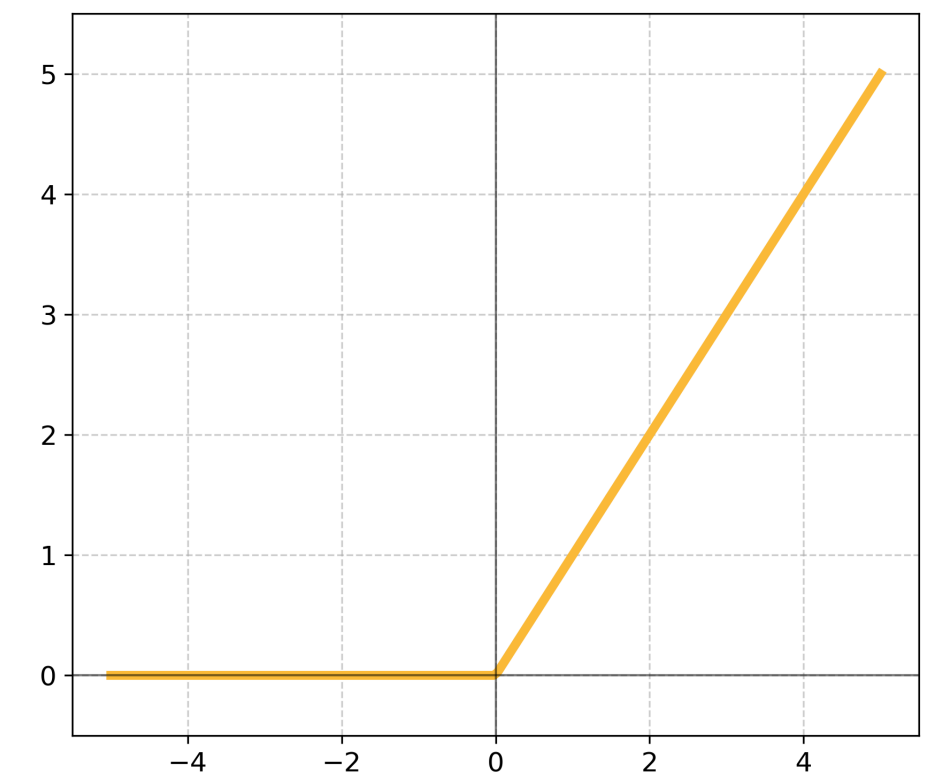
Tanh

$$\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$$



ReLU

$$f(x) = \max(0, x)$$





А зачем вообще нужны нелинейности?

Что будет, если мы уберем функцию σ из нашей сети и оставим только умножение матриц?

$$W_3(W_2(W_1 \cdot X)) = (W_3 \cdot W_2 \cdot W_1) \cdot X = W_{new} \cdot X$$

- Из-за свойства ассоциативности матричного умножения произведение любого количества матриц дает просто одну новую матрицу.
- Нейросеть из 100 слоев без активаций будет математически эквивалентна простейшей линейной регрессии.
- Именно нелинейность позволяет сети искривлять пространство признаков и выучивать сложные закономерности.



Почему нейросети хорошо работают?

Теорема (А.Н. Колмогоров, 1957)

Каждая непрерывная функция $a(x)$, заданная на единичном кубе d -мерного пространства, представима в виде:

$$a(x) = \sum_{i=1}^{2d+1} \sigma_i \left(\sum_{j=1}^d f_{ij}(x_j) \right)$$

где $x = [x_1, \dots, x_d]^T$ — вектор описания объекта, функции σ_i и f_{ij} непрерывны.

- Функция определена только на **единичном кубе**. Требуется масштабирование признаков.
- Теорема не указывает конкретный вид функций, вопрос подбора архитектуры открыт.

Теорема (Дж. Цыбенко, 1989)

Пусть σ — любая непрерывная сигмоида. Тогда для любой непрерывной функции $f(x)$ и $\epsilon > 0$ существует нейросеть с **одним скрытым слоем**, такая что:

$$|f(x) - \hat{f}(x)| < \epsilon$$

Где под нейросетью понимается матричная модель:

$$\hat{f}(x) = \sigma(XW_1 + b_1)W_2 + b_2$$

$$W_1 \in \mathbb{R}^{d \times N}$$

$$W_2 \in \mathbb{R}^{N \times 1}$$



Обучение MLP

Мы хотим найти такие веса, при которых функция потерь L минимальна. Для этого используется итеративный шаг градиентного спуска:

$$w_{new} = w_{old} - \eta \nabla L(w_{old})$$

Вспомним логистическую регрессию. Там мы могли аналитически вывести формулу градиента функции потерь и сразу подставить её в код:

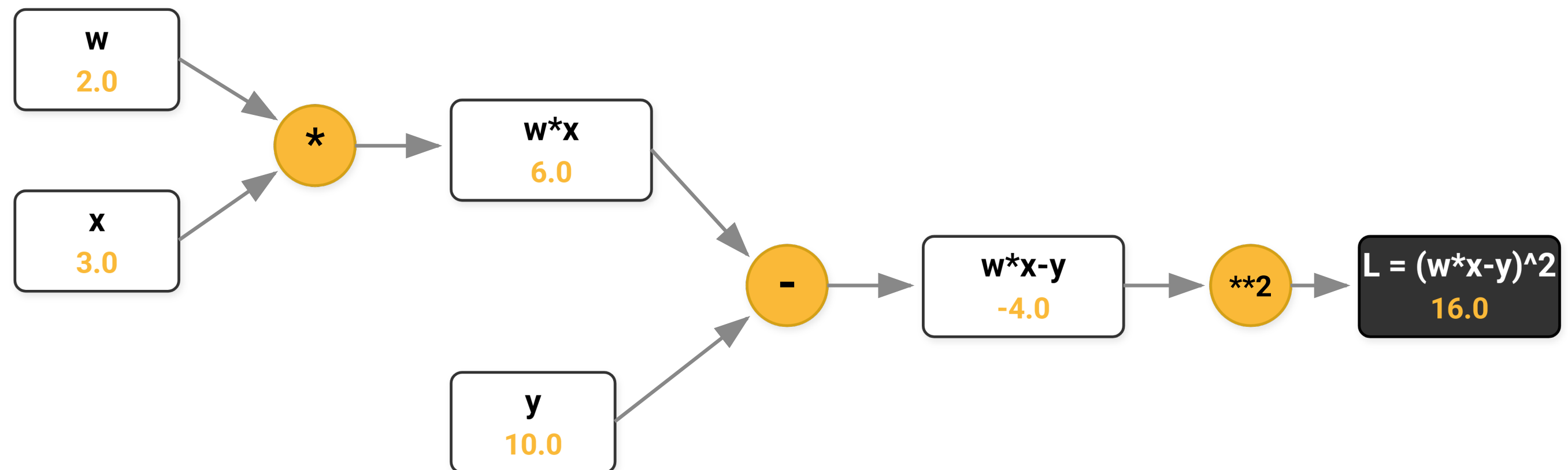
$$\nabla_w L = X^T (\hat{Y} - Y)$$

Однако для **многослойных сетей** этот подход не масштабируется. Вывести единую аналитическую формулу для сети из 100 слоев практически невозможно.



Граф вычислений

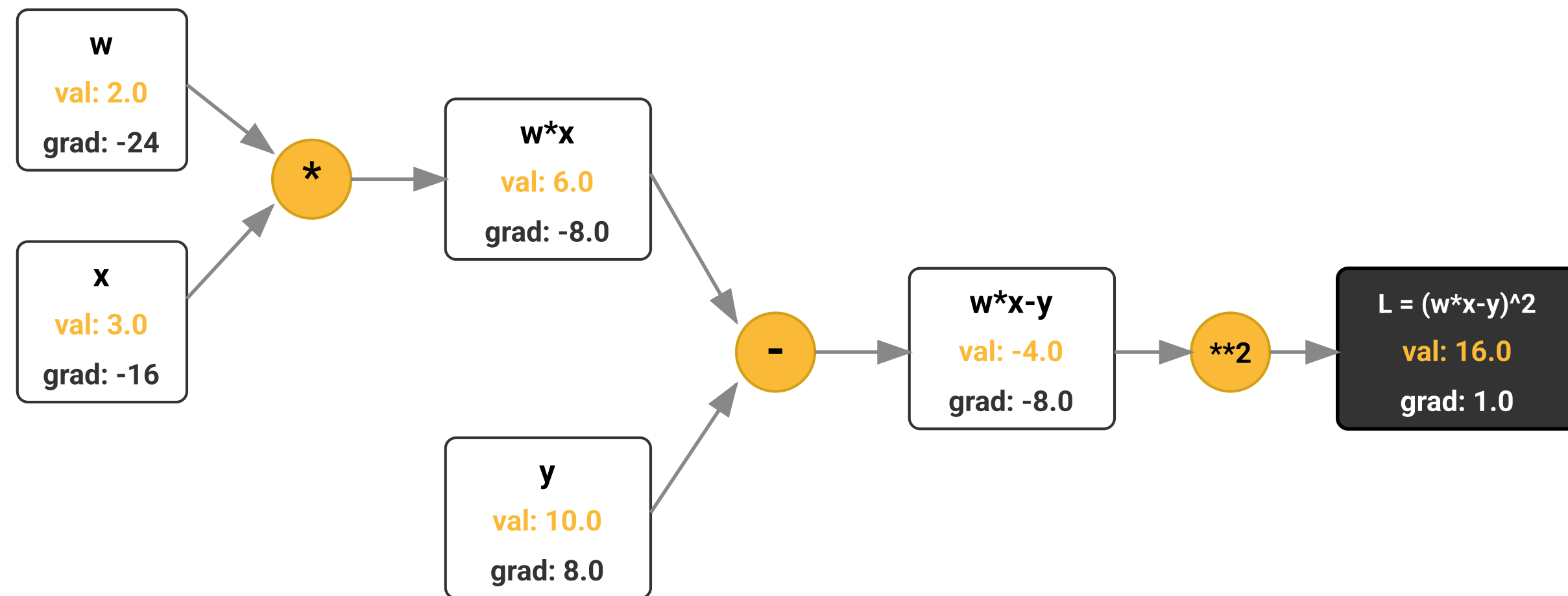
Возьмем более простой пример: $L = (w \cdot x - y)^2$. Подставим значения: $w = 2, x = 3, y = 10$. Разложим вычисление функции на элементарные операции и промежуточные переменные:





Backpropagation: Обратный проход

Двигаемся по графу в обратном направлении (от L). Вычисляем градиенты и сохраняем их в узлах:





Алгоритм обратного распространения ошибки

```
computation_graph.topsort()
computation_graph.zero_grad()

for node in computation_graph.nodes:
    if node.is_operation:
        inputs = node.inputs
        output = node.output
        for input in inputs:
            l_grad = compute_grad(output, input, node.op)
            input.grad += l_grad * output.grad
```

- **zero_grad()**: необходимо обнулять градиенты перед каждым шагом, так как фреймворки их суммируют.
- **Оператор +=** : позволяет корректно учитывать вклад параметра, если он встречается в графе несколько раз.

$$L = (w \cdot x - y)^2 + \lambda w^2$$

$$\frac{\partial L}{\partial w} = \frac{\partial (w \cdot x - y)^2}{\partial w} + 2\lambda w$$



Машинное обучение

Спасибо за внимание!

Вопросы?

Докладчик: Ваня Горохов